

ADR 0058 — Commit-path git spawns that run hooks are time-bounded, killed, and verified

Date: 2026-08-18 Status: Accepted — implemented

Extends ADR 0019 (detection, not exclusion, of outside git) and the M1.13/#66 execution policy; generalizes the `SIGN_TIMEOUT` contract `run_signed_tag` established in M2.21e (#239).
Supersedes nothing.

Context

72 (M2.19) requires that
"hooks cannot freeze the UI."
and
git commit

`git commit --amend` run repository hooks — arbitrary user code — through `run_git` → `git_output_for`, which awaits `cmd.output()` with no bound and no `kill_on_drop`. A `pre-commit` that blocks (waiting on input, a network call, a deadlocked script) therefore hangs the request forever, and — because the per-repository mutation guard (`coordinator::lock`) is held across `execute()` — queues every other write to that repository behind it. The client abandons the request after 60 s (`REQUEST_TIMEOUT_MS`, `crates/git-vista/src/api.rs`) with a message that, until this milestone,

blamed an SSH tunnel this deployment does not have. Hooks genuinely run in production: ADR 0029 rejected blocking them, and no production policy constructor yields `HookMode::Blocked`.

The mechanism for a bounded spawn already existed:

`git_output_bounded` (`crates/git-vista-server/src/git_cmd.rs`) — timeout + severed stdin + `kill_on_drop`, returning a typed `BoundedOutput::{Completed, TimedOut}` — built for signed tags, where a 10 s bound (`SIGN_TIMEOUT`) plus a bounded post-kill state read plus honest wording is the proven contract (`run_signed_tag`, `planner.rs`).

Decision

1. **Per-operation policy over the existing mechanism.**

`CommitOnHead` and `AmendCommit` execute through a new `run_git_hooked`, a thin wrapper over `git_output_bounded` with `HOOKED_GIT_TIMEOUT = 30 s`. No blanket timeout is added to `git_output_for`: reads are bounded by byte caps, and network operations are deliberately unbounded-with-cancellation.

`EmptyCommitOnBranch` (`commit-tree` + `update-ref`) stays on plain `run_git` — it runs no hooks by construction.

2. **30 seconds, one constant, not configurable.** It must clear the client's 60 s abandonment with room for the coordinator `Waiting` stage and the bounded verification read; it is generous for any hook that belongs on a commit button; and signing keeps its own tighter 10 s. A real >30 s hook is the trigger to make it a catalog field.

3. **Kill semantics ride the sandbox tier.** The `SIGKILL` lands on `bwrap`; the Strict tier's PID namespace — proven by the lifecycle suite to reap detached orphans — takes git and the hook down with it. Commit/amend are `Local` → Strict for every untrusted repo, and production has no trust grant. On a (today unreachable) Unsandboxed repo the bound limits our wait, not the hook's life — stated, as `run_signed_tag` states it.

4. **The timeout arm verifies, then tells the truth — no lock cleanup.** HEAD is re-read through `git_output_bounded` (never an unbounded spawn — the coordinator guard is still held). Three outcomes, each its own sentence in the refusal: HEAD unchanged (no commit landed), HEAD moved (a commit exists — inspect it, the kill raced a `post-commit` hang), or the verification read itself timed out (say so, name the command to run by hand). `.git/index.lock` **cleanup was designed but is *not* implemented — see "Corrected during implementation" below: it does not occur for this operation shape, so there is nothing to clean up.**
5. **The mutation guard is untouched.** It releases by the drop it always had; the fix is that execution now provably returns within the bound. Queued writers see a clean preflight, deterministically — measured, not assumed (see below).
5. **v1 is timeout-only.** `honours_cancellation` stays `false` for commit/amend: cancellation is for operations whose legitimate duration is unbounded, and after this decision commit's is not. If the budget ever grows past the client window, the extension point is the existing latch, not a new mechanism.
7. **The commit-path refusal stays untyped prose; only amend gets a typed kind.** `exec_amend_commit` already carries `AmendFailureKind` / `AmendCommitError` / `amend_refusal_body` (#223), so this adds `AmendFailureKind::HookTimedOut` to that closed set, and the frontend (`AmendRefusal::Timeout`) branches on it without regex-sniffing prose. `exec_commit_on_head` has no typed kind at all — it has always answered `(StatusCode, String)` — and inventing a wire DTO for one new case would be new API surface this slice does not need; its refusal is the same honest sentence, just as untyped prose. Typing commit's refusals (a `CommitFailureKind`, mirroring amend's) is a named follow-up, not scope creep now.

Corrected during implementation — the design's `index.lock` premise was wrong

The design doc this ADR implements (`design-docs/2026-08-17-hook-timeout-contract.md`, §3) argued at length that `git commit` holds `.git/index.lock` for the *duration* of `pre-commit`, and specified a three-guard removal path (tree provably dead, no lock at preflight, mtime inside the spawn window) so a killed hook could not brick the repository for `refuse_if_git_busy`.

Measured directly before writing any implementation code, both through the sandboxed `git_output_bounded` path and with a plain, unsandboxed `git commit` watched from a second shell: `.git/index.lock` **does not exist** while `pre-commit` or `post-commit` is running, sleeping or killed, for either `git commit` or `git commit --amend`. Git's own `prepare_to_commit` runs the hooks *before* it takes the index write-lock; by the time `post-commit` runs, the lock has already been taken and released (the index was written by rename). The design's whole premise for §3 — "SIGKILL leaves the lock file behind" — does not hold for this operation shape, on the git version this repository builds against.

Consequence: the three-guard removal path was never implemented, because there is nothing for it to remove.

`the_coordinator_lock_is_released_after_a_hook_timeout` (`planner/hook_timeout_suite.rs`) proves the stronger, simpler claim directly — a `CreateBranch` queued behind a timed-out, hook-hung commit on the same repository completes once the guard releases, with no lock-file special-casing on either side. If a future git version, hook shape, or platform is ever found to leave `index.lock` behind after this kill, the three-guard design above is already written and ready to implement; nothing observed during this milestone required it.

Alternatives considered

- **A blanket timeout in `git_output_for`**. Rejected: one value cannot serve fetch (minutes, cancellable) and plumbing (milliseconds) and hooks (seconds-tens of seconds); it would silently reinterpret every existing call site.
- **Blocking hooks (`core.hooksPath= empty dir`)**. Already rejected by ADR 0029; a commit whose hooks silently didn't run is a lie about what the repository's owner configured.
- **Making commit cancellable via `git_streamed_for now`**. Rejected for v1: real cost (a third execution lane, doubled post-kill obligations) against a 30 s worst case.
- **A conditional "this repository has a pre-commit hook" sentence in the timeout refusal**, gated on `rejectable_hook_present`. Rejected during implementation: that probe itself runs an unbounded `run_git` call (`rev-parse --git-path hooks` , then a filesystem stat) — calling it from the timeout arm, still inside the coordinator guard, on a repository that just proved a git child can block, would reintroduce the exact bug this ADR closes. The refusal states only what the bounded HEAD re-read actually found.
- **Typing `/api/commit`'s refusal to match amend's `AmendFailureKind`**. Deferred, not rejected: commit has no typed refusal kind today for *anything*, and adding one wire DTO for one new case is a bigger surface change than this slice's scope. Named as a follow-up alongside merge/checkout/rebase below.
- **`index.lock` guard-and-remove (the design's original §3)**. Superseded by measurement — see "Corrected during implementation" above — not by a judgment call.

Consequences

- A hanging hook costs at most `HOOKED_GIT_TIMEOUT` (30 s) of held guard and answers inside the client's 60 s window with a truthful refusal; the repository stays writable afterwards — proven by a real queued `CreateBranch` completing, not merely asserted.
- The repo may legitimately be left mid-operation (a `post-commit` hang commits first); the response says which state was observed rather than guessing — the same posture as the signed-tag timeout.
- `MergeBranch`, `CheckoutBranch`, and `RebaseOntoBase` also run hooks and remain unbounded; they are the named follow-up consumers of `run_git_hooked`.
- `/api/commit`'s refusal remains untyped prose; giving it a `CommitFailureKind` symmetrical to `AmendFailureKind` is a named follow-up, not done here.
- The 30 s constant is a bet on hook size; the refusal text names the actual bound it ran under (`{:?}` of `hooked_git_timeout()`, so a test's shrunk override prints truthfully too), so the first hook that legitimately needs longer is self-diagnosing rather than silently wrong.
- The `.git/index.lock` guard-and-remove design is documented above but unbuilt; if a future finding shows the lock *can* survive a killed commit/amend hook, that design is ready to implement rather than re-derive.

Where this is implemented

- `HOOKED_GIT_TIMEOUT`, `hooked_git_timeout()`, `run_git_hooked` — `crates/git-vista-server/src/planner.rs`, immediately after `run_git`.

- The timeout arm in `exec_commit_on_head` / `exec_amend_commit`, and the shared `HookTimeoutHeadCheck` / `check_head_after_hook_timeout` / `hook_timeout_message` — `crates/git-vista-server/src/planner.rs`.
- `AmendFailureKind::HookTimedOut` — `crates/git-vista-protocol/src/dto.rs`.
- `AmendRefusal::Timeout` and its dialog copy — `crates/git-vista/src/features/dialogs/commit.rs`.
- Tests — `crates/git-vista-server/src/planner/hook_timeout_suite.rs`: a sleeping `pre-commit` that times out and answers honestly, the coordinator guard releasing for a queued `CreateBranch`, a sleeping `post-commit` that reports the commit that landed, and a positive control (no hooks, real 30 s bound) proving `run_git_hooked` is not a slower `run_git`.